MCE5919: Gaussian Process for Machine Learning and Signal Processing

Lecture 4

Wenqiang PU

Shenzhen Research Institute of Big Data & School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Jan. 28, 2026

Roadmap

- Explain what a Gaussian process (GP) means: a distribution over functions.
- Read a kernel $k(x, x')$ as a modeling assumption (smoothness, periodicity, etc.).
- Derive the GP regression posterior by conditioning a joint Gaussian.
- Interpret the predictive mean (best guess) and variance (uncertainty).
- Know the key practical facts: $\mathcal{O}(n^3)$ training and why Cholesky is used.

Notation Cheat Sheet

Symbol	Meaning
$\mathbf{X} = [x_1, \ldots, x_n]$	training inputs (often $n \times d$)
$\mathbf{y} = [y_1, \ldots, y_n]^\top$	noisy observations
$f(\cdot)$	latent (noise-free) regression function
$k(x, x')$	kernel / covariance function
$K(X, X) = \mathbf{K}$	Gram matrix, $[K]_{ij} = k(x_i, x_j)$
σ_n^2	observation noise variance
$X_*,\ \mathbf{y}_*$	test inputs / predicted outputs

Overview

1. **Function-Space View: Gaussian Process Regression**

2. **Decision Theory for Regression**

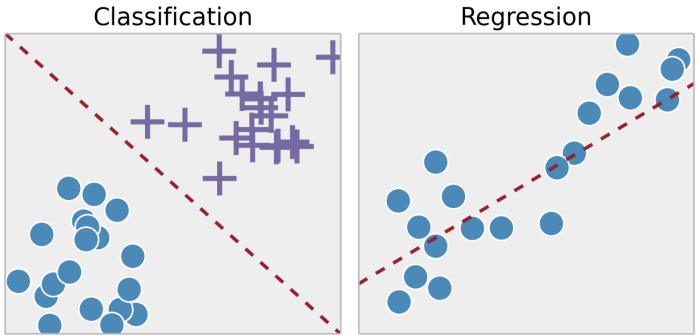
Where are we in the course?

- **Lecture 2: Linear regression (parametric)**
  - Output is mainly a point estimate $\hat{y}$.
- **Lecture 3: Bayesian linear regression + kernel trick**
  - We learned: a Bayesian model gives predictive distributions.
  - Kernel trick: nonlinear learning via similarity $k(x, x')$ without explicit features.
- **Today: Gaussian Process (GP) regression (nonparametric)**
  - GP is Bayesian regression directly over functions.
  - Kernel function is the core: it encodes smoothness, correlation, periodicity, etc.

Big Picture: What GP is used for

In this course, we view GP as an advanced Bayesian learning tool for:

- nonlinear regression
- classification



GP Definition

Definition I: Gaussian Process [Rasmussen, 2006]

A Gaussian process is a collection of random variables, any finite number of which have a joint Gaussian distribution.

Definition II: Gaussian Process [Theodoridis, 2015]

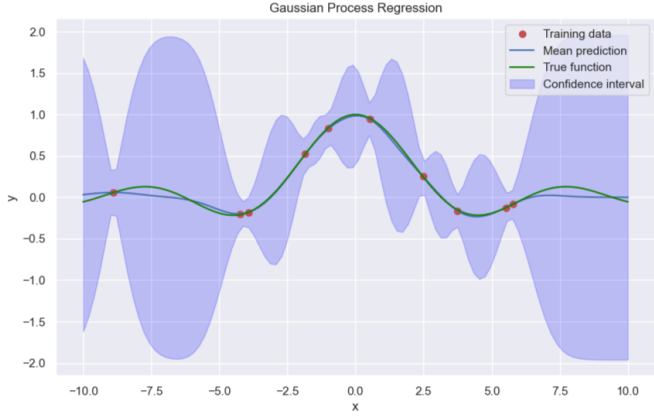
A random process, $f(\mathbf{x})$, is called a Gaussian process if and only if for any finite number of points, $\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N$, the associated joint probability density function (pdf), $p(f(\mathbf{x}_1), f(\mathbf{x}_2), \cdots, f(\mathbf{x}_N))$ is Gaussian.

Intuition: pick any finite set of input points, and the function values at those points behave like a multivariate Gaussian vector.

GP Definition

- Gaussian Process can be 'regarded' as distribution over functions

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$$



GP Definition

- A real GP $f(\mathbf{x})$, is completely specified by its mean function $m(\mathbf{x})$ and its covariance function/kernel function $k(\mathbf{x}, \mathbf{x}')$ as

$$\begin{aligned} m(\mathbf{x}) &= \mathbb{E}[f(\mathbf{x})], \\ k\left(\mathbf{x}, \mathbf{x}'\right) &= \mathbb{E}\left[\left(f(\mathbf{x}) - m(\mathbf{x})\right)\left(f\left(\mathbf{x}'\right) - m\left(\mathbf{x}'\right)\right)\right]. \end{aligned}$$

- We express a GP as

$$f(\mathbf{x}) \sim \mathcal{GP}\left(m(\mathbf{x}), k\left(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta}_h\right)\right).$$

Kernel Functions

- Choosing a kernel is choosing what kinds of functions you believe are likely.
- Different kernels $\Rightarrow$ different notions of similarity and smoothness.
- Some elementary kernel functions are:
  1. Linear kernel (with $\boldsymbol{\theta}_h = \sigma_0$)

$$k\left(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta}_h\right) = \sigma_0 + \mathbf{x}^T \mathbf{x}'$$

  2. Squared Exponential (SE)/Gaussian kernel (with $\boldsymbol{\theta}_h = [\sigma_s^2, \ell_s]^T$)

$$k\left(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta}_h\right) = \sigma_s^2 \exp\left(-\frac{||\mathbf{x} - \mathbf{x}'||^2}{2\ell_s^2}\right)$$

- **Rule of thumb:** start simple (e.g., SE), then improve based on data and residuals.

Kernel Function

Practical question: how do we choose $k(x, x')$?

- Use domain knowledge: smooth trend? periodicity? rough signals?
- Start simple, then add structure (sum/product kernels) if needed.
- Compare choices using validation or marginal likelihood.

GP Example

Consider finite set of inputs $x_1, \ldots, x_n$:

- Pick any finite set $x_1, \ldots, x_n$
- Then $\mathbf{f} = [f(x_1), \ldots, f(x_n)]$ is multivariate Gaussian

$$\mathbf{f} \sim \mathcal{N}(\mathbf{m}, \mathbf{K})$$

where $\mathbf{m} = [m(x_1), \ldots, m(x_n)]$, $K_{ij} = k(x_i, x_j)$

Key Observation: $\mathbf{f}$ is a random vector, $f(\cdot)$ is a random function!

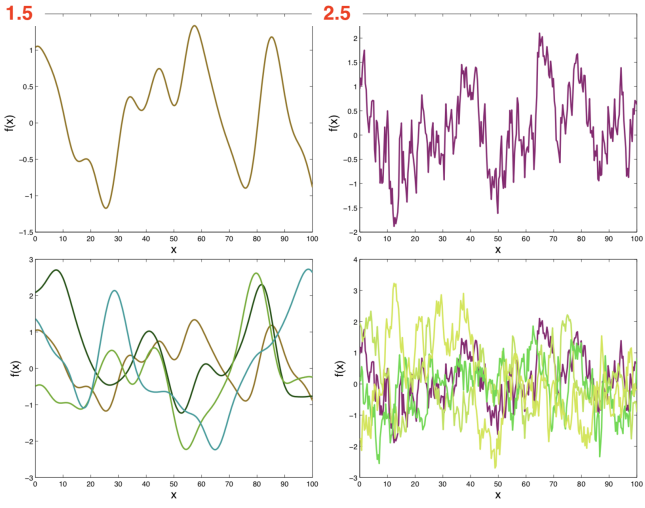


Figure: Kernel function has good interpretability!

GP Regression: Generative Model

We consider a latent function model

- **Function Class:** $f(\cdot)$ is drawn from a zero-mean GP prior with $k(\cdot,\cdot;\boldsymbol{\theta}_h)$

$$f(\cdot) \sim \mathcal{GP}(0,\ k(\cdot,\cdot;\boldsymbol{\theta}_h))$$

- **Data Generation:** The general regression model

$$y = f(\mathbf{x}) \ + \varepsilon,$$

  where the noise term (optional) is i.i.d. Gaussian with variance σ_n^2

- **Observation:** Consider one inputs $\mathbf{x}$

$$\mathbf{y} \mid f \sim \mathcal{N}(f,\ \sigma_n^2), \qquad \text{where } f \sim \mathcal{N}(m(\mathbf{x}), k(\mathbf{x},\mathbf{x};\boldsymbol{\theta}_h))$$

The set of all unknown parameters is denoted by $\boldsymbol{\theta} \triangleq [\boldsymbol{\theta}_h, \sigma_n^2]$.

GP Regression: Generative Model

- Given a finite number of training input points $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_n]$.
- Together with the observations, $\mathbf{y} = [y_1, y_2, ..., y_n]$
- Obtain the training dataset $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$.
- The assumed Gaussian process boils down to a multivariate Gaussian variable, with

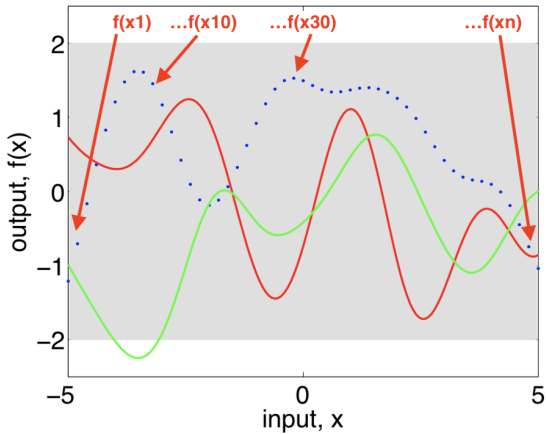
$$\mathbf{f} := [f(\mathbf{x}_1), f(\mathbf{x}_2), ..., f(\mathbf{x}_n)]^T \sim \mathcal{N}(\mathbf{m}, \mathbf{K})$$

where $\mathbf{m} = [m(\mathbf{x}_1), \ldots, m(\mathbf{x}_n)]$, $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$

Remark: GP is like an infinite-dimensional Gaussian (but we only ever use finite slices)

Gaussian Distribution $\leftrightarrow$ Gaussian Process

- Gaussian process: $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$
- Gaussian distribution: $\mathbf{f} \sim \mathcal{N}(\mathbf{m}, \mathbf{K})$



Marginal Likelihood $p(\mathbf{y} \mid X)$: Setup

Marginal likelihood (evidence) integrates out the latent function values $\mathbf{f}$:

$$p(\mathbf{y} \mid X, \boldsymbol{\theta}) = \int p(\mathbf{y} \mid \mathbf{f}, X, \boldsymbol{\theta})\, p(\mathbf{f} \mid X, \boldsymbol{\theta})\, d\mathbf{f}.$$

For GP regression (zero-mean prior for simplicity):

$$\begin{aligned} \mathbf{f} \mid X &\sim \mathcal{N}(\mathbf{0},\, K), \qquad K \equiv K(X, X), \\ \mathbf{y} \mid \mathbf{f}, X &\sim \mathcal{N}(\mathbf{f},\, \sigma_n^2 I). \end{aligned}$$

Marginal Likelihood $p(\mathbf{y} \mid X)$: Fast Derivation

Define the noisy covariance matrix

$$K_y \triangleq K + \sigma_n^2 I.$$

Write the model as a sum of independent Gaussians:

$$\mathbf{y} = \mathbf{f} + \varepsilon, \qquad \mathbf{f} \sim \mathcal{N}(\mathbf{0}, K), \qquad \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma_n^2 I), \qquad \varepsilon \perp\!\!\!\perp \mathbf{f}.$$

Hence covariance adds:

$$\mathbf{y} \mid X \sim \mathcal{N}(\mathbf{0},\, K + \sigma_n^2 I) = \mathcal{N}(\mathbf{0},\, K_y).$$

So the evidence is simply

$$\boxed{p(\mathbf{y} \mid X, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y} \mid \mathbf{0},\, K_y).}$$

Marginal Likelihood $p(\mathbf{y} \mid X)$: Integration Details

Start from the integral (treating it as a function of $\mathbf{f}$):

$$p(\mathbf{y} \mid X) = \int \mathcal{N}(\mathbf{y} \mid \mathbf{f}, \sigma_n^2 I)\, \mathcal{N}(\mathbf{f} \mid \mathbf{0}, K)\, d\mathbf{f}$$

$$\propto \int \exp\Big(-\tfrac{1}{2} \underbrace{\tfrac{1}{\sigma_n^2}(\mathbf{y}-\mathbf{f})^\top(\mathbf{y}-\mathbf{f})}_{\text{likelihood term}} \Big) \exp\Big(-\tfrac{1}{2} \underbrace{\mathbf{f}^\top K^{-1}\mathbf{f}}_{\text{prior term}} \Big)\, d\mathbf{f}.$$

Expand and collect terms in $\mathbf{f}$:

$$\frac{1}{\sigma_n^2}(\mathbf{y}-\mathbf{f})^\top(\mathbf{y}-\mathbf{f}) + \mathbf{f}^\top K^{-1}\mathbf{f} = \mathbf{f}^\top\Big(K^{-1} + \sigma_n^{-2} I\Big)\mathbf{f} - 2\sigma_n^{-2}\mathbf{y}^\top\mathbf{f} + \sigma_n^{-2}\mathbf{y}^\top\mathbf{y}.$$

Let $\Lambda \triangleq K^{-1} + \sigma_n^{-2} I$, $\Sigma \triangleq \Lambda^{-1}$, $\boldsymbol{\mu} \triangleq \Sigma\, \sigma_n^{-2}\mathbf{y}$. Then

$$\mathbf{f}^\top\Lambda\mathbf{f} - 2\sigma_n^{-2}\mathbf{y}^\top\mathbf{f} = (\mathbf{f}-\boldsymbol{\mu})^\top\Lambda(\mathbf{f}-\boldsymbol{\mu}) - \boldsymbol{\mu}^\top\Lambda\boldsymbol{\mu}.$$

Marginal Likelihood $p(\mathbf{y} \mid X)$: Log-marginal Likelihood

The integral over $\mathbf{f}$ becomes a normalized Gaussian times constants, yielding

$$\boxed{p(\mathbf{y} \mid X, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y} \mid \mathbf{0},\, K_y).}$$

Since $\mathbf{y} \mid X \sim \mathcal{N}(\mathbf{0}, K_y)$, we have

$$\begin{aligned}\log p(\mathbf{y} \mid X, \boldsymbol{\theta}) &= \log \mathcal{N}(\mathbf{y} \mid \mathbf{0}, K_y) \\ &= -\frac{1}{2}\mathbf{y}^\top K_y^{-1}\mathbf{y} \;-\; \frac{1}{2}\log|K_y| \;-\; \frac{n}{2}\log(2\pi).\end{aligned}$$

Prediction: What do we want?

- Not primarily interested in drawing random functions from the prior distribution.
- Goal: given $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$ and test inputs

$$\mathbf{X}_* = [\mathbf{x}_1^*, \mathbf{x}_2^*, ..., \mathbf{x}_{n_*}^*]$$

compute a predictive distribution

$$p(\mathbf{y}_* \mid \mathcal{D}, \mathbf{X}_*; \boldsymbol{\theta}).$$

- This gives both a prediction (mean) and uncertainty (variance).

Key Step: Build a Joint Gaussian

- A GP guarantees that any finite set of function values is jointly Gaussian.
- With Gaussian noise, the observed outputs are also jointly Gaussian.
- Concretely,

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{y}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K(X,X) + \sigma_n^2 \boldsymbol{I}_n & K(X,X_*) \\ K(X_*,X) & K(X_*,X_*) + \sigma_n^2 \boldsymbol{I}_{n_*} \end{bmatrix}\right),$$

where

- $K(X, X_*)$ is an $n \times n_*$ covariance matrix evaluated at all training and test points.
- $K(X_*, X)$, $K(X, X)$, $K(X_*, X_*)$ that are of size $n_* \times n$, $n \times n$, and $n_* \times n_*$, respectively.
- $K(X_*, X) = K(X, X_*)^T$.

Conditioning a Joint Gaussian (the only trick you need)

To get $p(\mathbf{y}_* \mid \mathcal{D}, X_*; \boldsymbol{\theta})$, we condition a joint Gaussian.

Conditional Gaussian Distribution

If the two random variables $\boldsymbol{x}$ and $\boldsymbol{y}$ are jointly multivariate Gaussian as

$$p(\boldsymbol{x}, \boldsymbol{y}) \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}_x \\ \boldsymbol{\mu}_y\end{bmatrix}, \begin{bmatrix}\Sigma_{xx} & \Sigma_{xy} \\ \Sigma_{yx} & \Sigma_{yy}\end{bmatrix}\right),$$

then the conditional is also Gaussian:

$$p(\boldsymbol{x}|\boldsymbol{y}) \sim \mathcal{N}(\boldsymbol{\mu}_{x|y}, \Sigma_{x|y}), \qquad p(\boldsymbol{y}|\boldsymbol{x}) \sim \mathcal{N}(\boldsymbol{\mu}_{y|x}, \Sigma_{y|x}),$$

where

$$\boldsymbol{\mu}_{x|y} = \boldsymbol{\mu}_x + \Sigma_{xy}\Sigma_{yy}^{-1}(\boldsymbol{y} - \boldsymbol{\mu}_y), \qquad \boldsymbol{\mu}_{y|x} = \boldsymbol{\mu}_y + \Sigma_{yx}\Sigma_{xx}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_x),$$
$$\Sigma_{x|y} = \Sigma_{xx} - \Sigma_{xy}\Sigma_{yy}^{-1}\Sigma_{yx}, \qquad \Sigma_{y|x} = \Sigma_{yy} - \Sigma_{yx}\Sigma_{xx}^{-1}\Sigma_{xy}.$$

GP Regression Posterior

Applying the above result to our GP regression model, we thus obtain

$$p(\mathbf{y}_*|\mathcal{D}, X_*; \boldsymbol{\theta}) \sim \mathcal{N}\left(\bar{\mathbf{m}}, \bar{V}\right),$$

where

$$\bar{\mathbf{m}} \triangleq K(X_*, X)\left[K(X, X) + \sigma_n^2 I_n\right]^{-1}\mathbf{y},$$

$$\bar{V} \triangleq K(X_*, X_*) + \sigma_n^2 I_{n_*} - K(X_*, X)\left[K(X, X) + \sigma_n^2 I_n\right]^{-1} K(X, X_*).$$

GP Regression Posterior

For the simplest case where there is only one test input $\mathbf{x}_*$, the posterior distribution is

$$p(y_*|\mathcal{D}, \mathbf{x}_*; \boldsymbol{\theta}) \sim \mathcal{N}\left(\bar{m}, \bar{v}\right),$$

where the posterior mean and variance are respectively

$$\bar{m} \triangleq \mathbf{k}_*^T \left[K(X,X) + \sigma_n^2 I_n\right]^{-1} \mathbf{y},$$

$$\bar{v} \triangleq k(\mathbf{x}_*, \mathbf{x}_*) + \sigma_n^2 - \mathbf{k}_*^T \left[K(X,X) + \sigma_n^2 I_n\right]^{-1} \mathbf{k}_*.$$

Here, $\mathbf{k}_* \triangleq [k(\mathbf{x}_*, \mathbf{x}_1), k(\mathbf{x}_*, \mathbf{x}_2), ..., k(\mathbf{x}_*, \mathbf{x}_n)]^T$ is the vector of covariances between the test point, $\mathbf{x}_*$, and the n training points in X.

Interpretation

Interpretations of the posterior mean and variance are:

- The posterior mean can be written as $\bar{m} = \boldsymbol{\beta}^T \mathbf{y}$, which is a linear combination of the observations, often referred as linear predictor.
- The posterior variance

$$\bar{v} \triangleq k(\mathbf{x}_*, \mathbf{x}_*) + \sigma_n^2 - \mathbf{k}_*^T \left[K(X, X) + \sigma_n^2 I_n\right]^{-1} \mathbf{k}_*$$

  is **prior uncertainty minus explained uncertainty**.
- Near many training points, uncertainty goes down; far away, it goes back up.

GP Regression in Practice (4-step recipe)

Given training data $\mathcal{D} = \{X, \mathbf{y}\}$:

1. Choose a kernel $k(\cdot, \cdot; \boldsymbol{\theta}_h)$ and noise level σ_n^2.
2. Build the matrix $K = K(X, X)$ and add noise: $K_y = K + \sigma_n^2 I$.
3. Solve $\boldsymbol{\alpha} = K_y^{-1}\mathbf{y}$ (numerically via Cholesky, not explicit inverse).
4. For test inputs X_*:
   - mean: $\bar{\mathbf{m}} = K(X_*, X)\,\boldsymbol{\alpha}$
   - variance: $\bar{V} = K(X_*, X_*) + \sigma_n^2 I - K(X_*, X)\,K_y^{-1}\,K(X, X_*)$

Cholesky Trick: Compute $K_y^{-1}\mathbf{y}$

In GP regression, we do not form K_y^{-1} explicitly. Instead we solve linear systems.

- Since $K_y = K + \sigma_n^2 I$ is symmetric positive definite, we can do Cholesky:

$$K_y = LL^\top, \quad L \text{ lower triangular.}$$

- To compute $\boldsymbol{\alpha} = K_y^{-1}\mathbf{y}$, solve

$$\begin{aligned} L\mathbf{v} &= \mathbf{y} \quad \text{(forward substitution)}, \\ L^\top \boldsymbol{\alpha} &= \mathbf{v} \quad \text{(back substitution)}. \end{aligned}$$

Why it matters: more stable than inversion; after one Cholesky ($\mathcal{O}(n^3)$), each solve is $\mathcal{O}(n^2)$.

Varying Hyperparameters

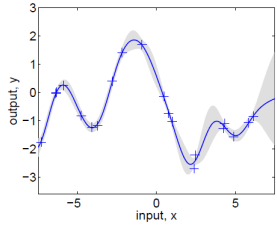
- Typically, the kernel functions that we selected have hyperparameters
- For example, the SE kernel function (1-D) is the following form

$$k_y(x_p, x_q) = \sigma_f^2 \exp\left(-\frac{(x_p - x_q)^2}{2\ell^2}\right) + \sigma_n^2 \delta_{pq}, \tag{1}$$

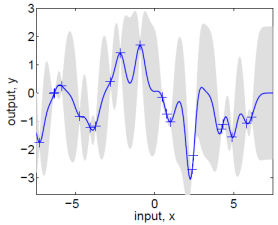
where the hyperparameters are

- ℓ: the length-scale
- σ_f^2: the signal variance
- σ_n^2: the independent noise variance

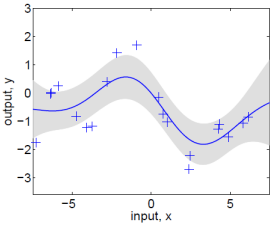
Note that the kernel function is denoted k_y as it is for the noisy targets y rather than for the underlying function f.



(a), $\ell = 1$



(b), $\ell = 0.3$



(c), $\ell = 3$

Figure: Panel (a): hyperparameters $(\ell, \sigma_f^2, \sigma_n^2) = (1, 1, 0.1)$. Panel (b): hyperparameters $(\ell, \sigma_f^2, \sigma_n^2) = (0.3, 1.08, 0.00005)$. Panel (c): hyperparameters $(\ell, \sigma_f^2, \sigma_n^2) = (3, 1.16, 0.89)$.

- We can see that the GP profile is largely due to the fitted hyperparameters.
- Can you tell me which set of hyperparameters is the best in your eyes? Why?

Model Training/Hyperparameter Tuning

- The predictive performance of GP regression depends on the goodness of a set of hyper-parameters tuned in the training phase.
- The following methods can be used to tune the GP hyper-parameters:

1. deterministic methods:
   - maximum likelihood (ML)
   - cross-validation (CV)
   - ...
2. stochastic methods:
   - hybrid Monte-Carlo (HMC)
   - Markov-Chain Monte-Carlo (MCMC)
   - ...

Deterministic ML based method is still most widely used.

Overview

1. Function-Space View: Gaussian Process Regression

2. Decision Theory for Regression

Decision Theory: From Distribution to a Point Prediction

- From GP regression, for a test input $\mathbf{x}_*$ we obtain a predictive distribution

$$p(y_* \mid \mathbf{x}_*, \mathcal{D}).$$

- In many applications we must output a single number $\hat{y}(\mathbf{x}_*)$ (a decision/action).
- What is optimal depends on the loss function

$$\mathcal{L}(y_*, \hat{y}) \quad \text{(penalty if truth is } y_* \text{ but we predict } \hat{y}\text{)}.$$

Bayes Decision Rule: Risk Minimization

- We cannot minimize $\mathcal{L}(y_*, \hat{y})$ directly because y_* is unknown.
- Instead, minimize the posterior expected loss (Bayes risk) under $p(y_* \mid \mathbf{x}_*, \mathcal{D})$:

$$R_{\mathcal{L}}(\hat{y} \mid \mathbf{x}_*) = \mathbb{E}_{y_* \sim p(\cdot \mid \mathbf{x}_*, \mathcal{D})}\big[\mathcal{L}(y_*, \hat{y})\big] = \int \mathcal{L}(y_*, \hat{y})\, p(y_* \mid \mathbf{x}_*, \mathcal{D})\, dy_*.$$

- The Bayes-optimal point prediction is

$$\hat{y}_{\text{opt}}(\mathbf{x}_*) = \arg\min_{\hat{y}} \; R_{\mathcal{L}}(\hat{y} \mid \mathbf{x}_*).$$

Common Losses $\Rightarrow$ Optimal Point Predictor

- Squared loss $\mathcal{L}(y_*, \hat{y}) = (y_* - \hat{y})^2$
  - $\hat{y}_{\text{opt}} = \mathbb{E}[y_* \mid \mathbf{x}_*, \mathcal{D}]$ (predictive mean).
- Absolute loss $\mathcal{L}(y_*, \hat{y}) = |y_* - \hat{y}|$
  - $\hat{y}_{\text{opt}} = \text{median}\big(p(y_* \mid \mathbf{x}_*, \mathcal{D})\big)$.
- 0-1 loss (classification-style) $\mathcal{L}(y_*, \hat{y}) = \mathbf{1}\{y_* \neq \hat{y}\}$
  - $\hat{y}_{\text{opt}} = \arg\max_y \; p(y \mid \mathbf{x}_*, \mathcal{D})$ (mode / MAP).
- Asymmetric costs often lead to quantiles (e.g., predict the τ-quantile).
- If $p(y_* \mid \mathbf{x}_*, \mathcal{D})$ is Gaussian, mean $=$ median $=$ mode.

Summary

- GP = distribution over functions; kernels encode assumptions.
- Prediction is conditioning a joint Gaussian $\Rightarrow$ closed-form mean and variance.
- Mean = best guess; variance = uncertainty that adapts to data.
- Standard GP scales as $\mathcal{O}(n^3)$ (matrix factorization)

The End